

1. Update proc.h

```
// proc.h (add near top)
#define DEFAULT_PRIORITY 20

// inside struct proc { ... }
int priority;
```

2. Update proc.c

```
// proc.c (inside allocproc(), after p->pid = nextpid++; )

p->priority = DEFAULT_PRIORITY;

// proc.c — replace existing scheduler() with this
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // enable interrupts on this processor.
        sti();

        struct proc *highest_priority_p = 0;
        int highest_priority = 1000000; // big number so any real priority is lower

        acquire(&ptable.lock);
        // scan process table for RUNNABLE and pick the lowest priority value
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;
            if(p->priority < highest_priority){
                highest_priority = p->priority;
                highest_priority_p = p;
            }
        }

        if(highest_priority_p != 0){
            // run the selected process
            p = highest_priority_p;
            c->proc = p;
            switchvm(p);
            p->state = RUNNING;
            swtch(&(c->scheduler), p->context);
            switchkvm();
            // when process yields/blocks/exits it will come back here
            c->proc = 0;
        }
    }
}
```

```

    release(&ptable.lock);
}
}

```

3. sysproc.c

```

int
sys_setpriority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
        return -1;

    struct proc *cur = myproc();
    cur->priority = priority;
    return 0;
}

```

4. syscall.h

```

#define SYS_setpriority 33

```

5. syscall.c

```

// syscall.c (add near other externs)
extern int sys_setpriority(void);

// then inside the syscalls table:
[SYS_setpriority] sys_setpriority,

```

6. user.h

```

int setpriority(int);

```

7. usys.S

```

SYSCALL(setpriority)

```

8. create prioritytest.c

```

// prioritytest.c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
    int pids[5];
    int priorities[5] = {5, 15, 25, 35, 45}; // assign priorities of your choice

    printf(1, "Starting priority scheduling test with 5 processes...\n");
}

```

```

for (int i = 0; i < 5; i++) {
    pids[i] = fork();
    if (pids[i] == 0) {
        // Child process
        setpriority(priorities[i]);
        printf(1, "Child %d (pid %d) started with priority %d\n",
            i+1, getpid(), priorities[i]);

        // Busy loop to simulate CPU work
        for (volatile int j = 0; j < 500000000; j++) {}

        printf(1, "Child %d (pid %d) finished.\n", i+1, getpid());
        exit();
    }
}

// Parent waits for all children
for (int i = 0; i < 5; i++) {
    wait();
}

printf(1, "Priority scheduling test with 5 processes complete.\n");
exit();
}

```

9. Makefile

_prioritytest

```

QEMU
Machine View

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFCB050+1EF0B050 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
12340920$prioritytest
Starting priority scheduling test with 5 processes...
Child 1 (pid 4) started with priority 5
Child 1 (pid 4) finished.
Child 2 (pid 5) started with priority 15
Child 2 (pid 5) finished.
Child 3 (pid 6) started with priority 25
Child 3 (pid 6) finished.
Child 4 (pid 7) started with priority 35
Child 4 (pid 7) finished.
Child 5 (pid 8) started with priority 45
Child 5 (pid 8) finished.
Priority scheduling test with 5 processes complete.
12340920$

```